

Pipeline CI/CD

Automação inteligente para projetos de Machine Learning agrícola com GitHub
Actions e Docker



Por que usar GitHub Actions?

Para criar um pipeline de CI/CD para o seu projeto de Machine Learning agrícola, utilizaremos o GitHub Actions. Ele é a escolha natural, pois já está integrado ao seu repositório e permite automatizar todo o processo que você fez manualmente até agora.

- ✔ **Objetivo central:** Sempre que você der um `git push`, o GitHub vai automaticamente testar seu código, criar a imagem Docker e enviá-la para o seu Docker Hub.

Integrado ao Repositório

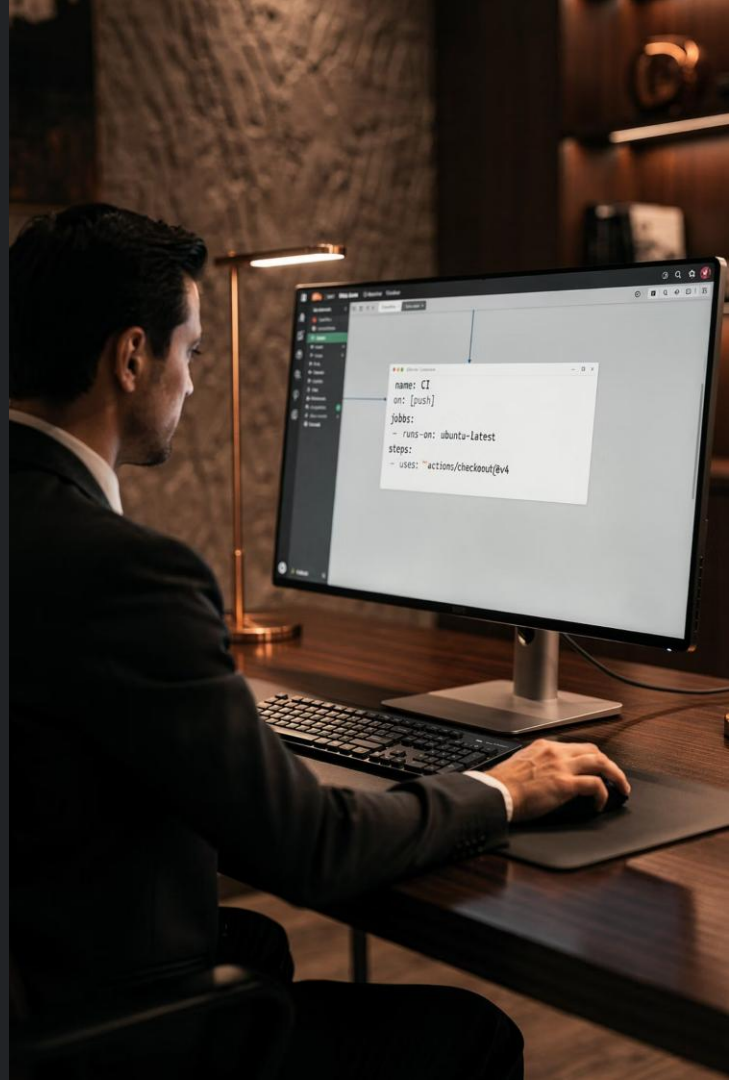
Sem ferramentas externas — tudo dentro do GitHub

Automação Total

Do código ao deploy sem intervenção manual

Gratuito para Projetos

Plano generoso para repositórios públicos e privados



O que o Pipeline CI/CD faz?

O arquivo `.github/workflows/main.yaml` age como um gerente de produção digital. Sempre que você faz um `git push`, ele executa quatro missões críticas de forma invisível e padronizada:

1

Provisionamento de Ambiente (Runner)

O GitHub "aluga" instantaneamente uma máquina virtual limpa com Ubuntu Linux. Isso garante que o build não herde "lixo" ou configurações específicas do seu computador pessoal. É o ambiente neutro ideal.

2

Integração Contínua (CI)

Checkout: O robô baixa a versão mais recente do seu código.
Validação: Verifica se o Dockerfile está correto e se todas as dependências do Python (Pandas, Scikit-Learn) podem ser instaladas sem conflitos. Se algo quebrar, o pipeline para e te avisa.

3

Entrega Contínua (CD)

Docker Build: O robô executa a "receita" do seu Dockerfile, empacotando o script de agricultura de precisão e o modelo Random Forest dentro de uma imagem imutável. **Segurança:** Utiliza as chaves dos Secrets para autenticar no Docker Hub sem expor sua senha.

4

Distribuição Global (Deployment)

Push: A imagem finalizada é enviada para o Docker Hub, disponível globalmente para qualquer pessoa com acesso ao repositório.

Disponibilidade Global e Valor para a Scania

A partir do momento do deploy, qualquer pessoa na Scania ou no mundo com acesso ao repositório pode rodar o modelo atualizado apenas digitando `docker pull`. O "robô" garante que a versão `latest` seja sempre a mais nova e funcional.

Por que isso é vital para a Scania e para o Mercado?



Eliminação do Erro Humano

Ninguém esquece de copiar um arquivo ou de instalar uma biblioteca. O robô faz sempre o mesmo processo, sem variações ou esquecimentos.



Velocidade (Time-to-Market)

Do código pronto à execução global em menos de 2 minutos. Agilidade que transforma a competitividade do time.



Rastreabilidade

Cada "bolinha verde" no GitHub é um registro histórico de quem alterou o quê, quando a imagem foi gerada e se ela passou nos testes.

O "Cofre" Digital

GitHub Secrets

Duração estimada: 30 minutos

Objetivo: Preparar o GitHub para conversar com o Docker Hub de forma segura e criptografada, sem expor suas credenciais no código.

Ação no Navegador (GitHub)

01

Navegar até: `Settings > Secrets and variables > Actions`

02

Clicar em New repository secret

03

Criar chave: `DOCKER_USERNAME` | Valor: `seu_usuario`

04

Criar chave: `DOCKER_PASSWORD` | Valor: `seu_token_gerado_no_hub`



Nunca coloque senhas diretamente no código. Use sempre os Secrets do GitHub para proteger suas credenciais.

Construindo a Esteira Automatizada

CI/CD

Objetivo da Etapa: Criar o arquivo de configuração que automatiza o trabalho braçal de build, teste e deploy.

Comando de Terminal — Criar Estrutura

```
PowerShell  
mkdir -p .github/workflows
```

 O GitHub só lê o arquivo de workflow se ele estiver exatamente no caminho `.github/workflows/main.yaml`. Qualquer variação no nome ou pasta e o pipeline não será ativado.

Conteúdo do arquivo `main.yaml`

O arquivo define três grandes blocos de instruções para o robô:

→ Login no Docker Hub

Usando os Secrets configurados anteriormente

→ Build da Imagem

Executa o Dockerfile e empacota o modelo

→ Push Automático

Envia a imagem para o Docker Hub com a tag `latest`

O Grande Deploy — Execução e Ciclo de Vida

EXECUÇÃO

Objetivo da Etapa: Ativar o pipeline e validar a portabilidade global do modelo de Machine Learning.

Comandos Sequenciais de Gatilho — O "Pulo do Gato"

PowerShell

```
# 1. Preparar os arquivos (Dockerfile e Scripts na raiz)
git add .

# 2. Documentar a versão
git commit -m "feat: ativando pipeline de automação global"

# 3. Enviar e disparar a máquina (Bolinha amarela no GitHub)
git push origin main
```

Passo 1 — Monitoramento

Abrir a aba Actions no GitHub logo após o push

Passo 2 — Verificação

Verificar o log do Build and Push em tempo real

Passo 3 — Confirmação

Aguardar a bolinha verde — sinal de sucesso total

Validação e Teste de Impacto

VALIDAÇÃO FINAL

Objetivo da Etapa: Provar que o modelo está disponível em qualquer lugar do mundo, rodando as previsões agrícolas de 2025–2029 a partir da imagem publicada.

Comando de Execução em qualquer terminal

PowerShell

```
# Puxar a versão que o "robô" acabou de criar
docker pull celiowillersouza/scania-devops:latest

# Rodar o modelo e ver as previsões de 2025-2029
docker run --rm celiowillersouza/scania-devops:latest
```

⚠ **Dica sobre Erros:** Se o build falhar com "Dockerfile not found", lembre-se que o Docker é cego para subpastas. O context: `.` define que ele só olha para a raiz do repositório.

📄 **O Valor do Machine Learning:** As previsões de *Produção Prevista (t/ha)* para 2025–2029 só são confiáveis porque o ambiente Docker garante que a biblioteca `scikit-learn` seja exatamente a mesma em todas as máquinas.

● A Bolinha Verde

No mundo DevOps, "Green is Good". O selo de sucesso no GitHub Actions é a certificação de que o código do aluno é profissional e está pronto para o mercado.

Código

Git Push

GitHub Actions

Docker Hub